

Acordo de Manutenibilidade e Engenharia de Software

Nome do Projeto: AgendaBeach - Agendamento de Quadras de Beach Tennis

Nome da Equipe ou Grupo: Gabriel Albani De Souza, Miguel Augusto Guedes, Murilo Enzo Watanabe, Nathalia Aline Berri Silva, Vinícius Henrique Da Silva.

Data da Elaboração: 07/08/2026

1. Identificação da Equipe e Papéis

Product Owner (PO): Nathalia Aline Berri Silva

Engenheiro de Requisitos: Nathalia Aline Berri Silva

Quality Assurance (QA): Gabriel Albani De Souza

Desenvolvedor Frontend: Murilo Enzo Watanabe

Desenvolvedor Backend: Vinícius Henrique Da Silva

DevOps: Miguel Augusto Guedes

2. Processo e Gestão de Débito Técnico (Responsável: Nathalia Aline Berri Silva)

2.1. Política de Registro de Débito Técnico

Todo atalho técnico, regra de negócio simplificada temporariamente ou pendência de refatoração identificada no AgendaBeach deve ser registrado como um card no backlog do Jira com a tag Débito Técnico. O registro deve conter a justificativa da decisão, a localização no código e o impacto potencial na manutenibilidade da aplicação.

2.2. Priorização e Pagamento da Dívida

A equipe destinará 15% do tempo útil de cada ciclo de desenvolvimento para a refatoração e pagamento dos débitos técnicos acumulados. Débitos que provoquem inconsistência na reserva de quadras, falhas na autenticação de usuários ou degradação do tempo de resposta da API de horários terão prioridade total de resolução no ciclo imediatamente seguinte.

3. Critérios de Qualidade: DoR e DoD (Responsável: Gabriel Albani De Souza e Nathalia Aline Berri Silva)

3.1. Definition of Ready — DoR (Pronto para Desenvolver)

Uma tarefa ou história do AgendaBeach só poderá ser puxada para desenvolvimento se cumprir 100% dos seguintes requisitos:

- História de Usuário padronizada: Formatada estritamente como "Como [cliente/administrador], eu quero [realizar uma ação] para que [obtenha um benefício]".
- Critérios de Aceite definidos: Comportamento esperado da tela e da API descrito sem ambiguidades (ex: "Não permitir cancelamento com menos de 2h de antecedência").
- Regras de Negócio e Exceções mapeadas: Regras de horário de pico, valores de tarifa e limites de agendamento por CPF devidamente especificadas.
- Mapeamento de Dependências: Modelagem do banco de dados (tabelas/FKs) e rotas da API backend identificadas.
- Prototipagem aprovada: Layout de tela e fluxo visual validados pelo PO para tarefas com interface frontend.

3.2. Definition of Done — DoD (Pronto para Entrega / Concluído)

Uma funcionalidade só receberá o status de "Concluída" (Done) no AgendaBeach se atender rigorosamente aos seguintes critérios de aceite de engenharia:

- Padrão de Código: Implementado conforme as diretrizes do guia de estilo da equipe (React.js no frontend e Java Spring Boot no backend).
- Code Review Aprovado: Código submetido via Pull Request e aprovado por pelo menos outro desenvolvedor/QA da equipe.
- Validação de QA: Testado pelo QA nos ambientes de testes, cobrindo o fluxo principal e cenários de exceção sem nenhum bug de severidade

alta ou crítica em aberto.

- Integração de Código: Código mesclado na branch principal de desenvolvimento (dev) sem conflitos de versionamento.
- Documentação Atualizada: Endpoints criados ou alterados devidamente documentados no Swagger/OpenAPI.

4. Estratégia de Testabilidade (Responsável: Gabriel Albani De Souza)

4.1. Cobertura e Níveis de Teste

Exemplo: O foco primário dos testes automatizados será nas regras de negócio da aplicação (backend/domain) e Antes de cada entrega oficial, o QA executará um roteiro de testes cobrindo o fluxo principal e as regras de negócio críticas.

4.2. Registro e Resolução de Erros (Bugs)

Exemplo: Todo bug identificado durante os testes deve ser documentado com: Passos esperados e passos obtidos sendo que Bugs de alta severidade bloqueiam o encerramento das tarefas

5. Guia de Estilo e Padrões de Código (Responsáveis: Murilo Enzo Watanabe e Vinícius Henrique Da Silva)

5.1. Convenções de Código (Style Guide)

- Linguagem:
 - Frontend: React.js.
 - Backend: Java com Spring Boot.
- Banco de Dados: Postgresql.
- Controle de Versão: GitHub.
- Nomenclatura de Variáveis/Funções: nmExemplo
- Nomenclatura de Arquivos: MeuArquivo.extensão
- Idioma do Código: exemplo: Inglês

5.2. Boas Práticas de Manutenibilidade

- Evitar duplicação de código criando funções e módulos reutilizáveis.
 - As funções/métodos devem possuir responsabilidade única.
 - Evitar aninhamento desnecessário.
 - Evitar uso de memória desnecessário (Ex: Criação de variáveis com apenas uma utilização).
 - Code Review obrigatório.
-

6. Fluxo de Versionamento e Pipeline de CI/CD (Responsável: Miguel Augusto Guedes)

6.1. Estratégia de Branches (Versionamento)

- main /master: Código estável e pronto para produção/entrega.
- dev: Código em integração para a entrega atual.
- Branches de Funcionalidade: Devem seguir o padrão: `feat/numero da tarefa/nome-da-funcionalidade` ou `fix/numero da tarefa/descrição`.

6.2. Padronização de Commits

- feat pra Nova funcionalidade.
- bug para Correção de bug.
- refactor para alteração (Manutenção Preventiva).
- docs para alterações na documentação.

6.3. Verificação Automática e Linters

- Ferramentas Configuradas: ESLint, Prettier e SonarQube
 - Regra de Integração: Exemplo: Nenhum Pull Request será aprovado se a verificação acusar erros críticos.
-

7. Compromisso da Equipe

Todos os membros da equipe leram, concordam e se comprometem a seguir as diretrizes estabelecidas neste documento para garantir a qualidade, manutenibilidade e entrega sustentável do software.

(x) Product Owner (PO)

(x) Engenheiro de Requisitos

(x) Quality Assurance (QA)

(x) Desenvolvedor Frontend

(x) Desenvolvedor Backend

(x) DevOps